

ConstruyeWeb

Diccionario de Datos Exhaustivo y Decisiones de Arquitectura

Versión 2.0

Desglose integral del modelo de base de datos relacional definido en `backend/schema_construyeweb.sql`, que abarca catálogo, inventario, e-commerce, compras, personal, cajas y facturación unificada. Análisis de normalización desde el inventario base (6,952 ítems), tipos de datos de precisión fija, auditoría inmutable de inventario (Kardex), control monetario multicanal, políticas de concurrencia y modelo transaccional.

Sistema:	Plataforma E-Commerce, Inventario y POS Ferretero
Motor de Base de Datos:	PostgreSQL 16+ (Cumplimiento ACID Estricto)
Versión del documento:	2.0 — incorpora la capa administrativa y de punto de venta (empleados, compras, cajas, facturación unificada, parámetros)
Autoría y Diseño BD:	Jonathan Manzanares (Modelado PostgreSQL & Frontend Next.js)
Backend y Negocio:	Elizabeth (Django REST Framework & Panel POS)
Fuente de Verdad DDL:	<code>backend/schema_construyeweb.sql</code> (564 líneas, 27 tablas , 3 vistas)
Fecha de Emisión:	Septiembre 2026

0. HISTORIAL DE CAMBIOS: VERSIÓN 2.0

0.1. Por qué existe la versión 2.0

La versión 1.0 del esquema resolvía muy bien el **catálogo, el inventario multisucursal y el flujo del e-commerce**, pero le faltaba la mitad administrativa de la plataforma. Concretamente, no existía dónde registrar: (a) una venta física de mostrador, (b) qué empleado la realizó y desde qué caja, (c) las compras de mercadería a proveedores, ni (d) el dinero en efectivo que entra y sale de cada caja. Sin estas piezas, el "control monetario" solo alcanzaba a los pedidos web y las ventas de las sucursales físicas quedaban fuera del modelo.

La versión 2.0 cierra esa brecha incorporando la capa administrativa que faltaba, de modo que la plataforma integra **dos sistemas sobre una única base de datos**: el e-commerce (Next.js) y el panel de inventario, POS y finanzas (Django).

0.2. Alcance de la versión 2.0

Cada módulo nuevo responde a una necesidad concreta del flujo de la ferretería. La siguiente tabla resume qué se agregó y por qué.

Necesidad del negocio	Solución en v2.0	Justificación
Saber quién opera el sistema y con qué cargo	<code>empleados</code> (+ <code>auth_user</code> de Django)	La contraseña y los permisos los gestiona Django (usuarios y grupos), por lo que no se duplican . <code>empleados</code> conserva solo el perfil de RR.HH.: cédula, salario, bonos, ingreso y jefe directo.
Cobrar en mostrador y cuadrar el efectivo	<code>cajas</code> , <code>sesiones_caja</code> , <code>movimientos_caja</code>	Se separa la terminal física del turno del cajero para soportar varias cajas por sucursal, y el efectivo se controla con asientos en lugar de un contador acumulado (ver Decisiones 10.1 y 10.2).
Facturar ventas físicas y web con un mismo criterio	<code>facturas</code> , <code>factura_items</code>	Una sola tabla para ambos canales, diferenciados por <code>canal</code> . Evita duplicar reportes, IVA y utilidad en dos modelos. Las líneas usan snapshot (igual que <code>pedido_items</code>) y guardan el costo para calcular la utilidad real.
Reponer mercadería y actualizar costos	<code>compras</code> , <code>compra_items</code>	Registran las órdenes a proveedores con su sucursal de destino; al recibirlas alimentan el inventario, el Kardex y el costo promedio.
Configurar reglas globales del negocio	<code>parametros</code>	Fila única y tipada: datos del negocio, moneda, IVA sugerido, tope de efectivo por caja y umbral general de stock bajo.
Atender clientes sin cuenta web	<code>clientes</code> (modificada)	Antes todo cliente tenía correo y contraseña. Ahora una sola tabla admite clientes web y de mostrador (ver Decisión 6.1).
Registrar cobros de ventas físicas y web en un mismo lugar	<code>pagos</code> (modificada)	Un pago puede pertenecer a un pedido, a una factura o a ambos, y una venta física admite pagos mixtos (ver Decisión 7.4).
Saber por qué cambió el stock	<code>movimientos_inventario</code> (modificada)	Cada movimiento apunta ahora al documento que lo originó (compra, factura o pedido) mediante llaves foráneas reales (ver Decisión 5.3).

0.3. Resumen de objetos añadidos y modificados

Objeto	Cambio	Sección
<code>parametros</code>	Tabla nueva	8.1
<code>empleados</code>	Tabla nueva	8.2
<code>compras</code> , <code>compra_items</code>	Tablas nuevas + ENUM <code>estado_compra</code>	9
<code>cajas</code> , <code>sesiones_caja</code> , <code>movimientos_caja</code>	Tablas nuevas + ENUMs <code>estado_sesion_caja</code> , <code>tipo_movimiento_caja</code>	10
<code>facturas</code> , <code>factura_items</code>	Tablas nuevas + ENUMs <code>canal_venta</code> , <code>estado_factura</code>	11
<code>clientes</code>	Email y contraseña opcionales; nuevas columnas <code>tipo_cliente</code> , <code>origen</code> ; 1 CHECK; 2 índices; ENUM <code>tipo_cliente</code>	6.1
<code>pagos</code>	<code>pedido_id</code> opcional; nuevas columnas <code>factura_id</code> , <code>sesion_caja_id</code> ; 3 CHECK; 2 índices	7.4

Objeto	Cambio	Sección
movimientos_inventario	Nuevas FK compra_id , factura_id , pedido_id , empleado_id ; 2 índices	5.3
v_alerta_stock_bajo , v_ventas_diarias	Vistas nuevas	12
Triggers updated_at	Extendidos a empleados , compras y parametros	13

0.4. Convención visual y validación de esta versión

En todo el documento, las secciones y tablas nuevas llevan la etiqueta **NUEVO v2.0** y las existentes que cambiaron **MODIFICADO v2.0**. Dentro de las tablas, las filas **nuevas** aparecen resaltadas en verde y las **modificadas** en azul (en las tablas de columnas además llevan una marca *nuevo* o *mod.*). El resto del contenido es el de la versión 1.0.

Validación: el script completo `schema_construyeweb.sql` se ejecutó sobre una base vacía de PostgreSQL 16.15 con `ON_ERROR_STOP` sin errores. Además se probó que las reglas nuevas se comportan como se documenta: rechazo de una segunda sesión abierta en la misma caja, de facturas POS sin sesión, de facturas e-commerce sin pedido, de pagos sin destino o con método inválido, y de clientes con contraseña pero sin correo; y el cálculo correcto de `subtotal` , `impuesto` , `diferencia` y de ambas vistas.

1. CONTEXTO DEL NEGOCIO Y NORMALIZACIÓN DEL INVENTARIO

El punto de partida del proyecto fue un inventario comercial plano contenido en el archivo `INVENTARIO_FERRETERIA_CONSTRUYE.xlsx` con 6,952 filas de productos. Dicho archivo reflejaba los vicios clásicos de los reportes tabulares operativos:

- **Desnormalización severa:** Categoría, Subcategoría y Clase estaban forzadas en 3 columnas estáticas. De las 6,952 filas, exactamente 6,945 productos no tenían categoría asignada.
- **Monocódigo de barras:** Una sola columna para código de barras, imposibilitando distinguir entre la venta de un tornillo suelto, una caja de 100 unidades o un bulto mayorista.
- **Puntos de Venta (PDV) como texto:** No existía aislamiento de stock por sucursal física; las existencias estaban centralizadas en un único número plano sin trazabilidad territorial.
- **Sobrescritura de precios:** Los costos y precios de venta eran celdas modificables directamente, borrando el historial de costos de adquisición pasados.

DECISIÓN DE DISEÑO 1.1: MIGRACIÓN A TERCERA FORMA NORMAL (3NF) Y NULLABILITY ESTRATÉGICO

El esquema descompuso el reporte plano en 18 tablas interrelacionadas (27 con la versión 2.0). Crucialmente, el atributo `productos.categoria_id` se definió como `NULLABLE`. Si se hubiese definido como `NOT NULL`, la importación de los 6,945 artículos sin categorizar habría fallado por violación de integridad. Esto permite importar todo el catálogo de inmediato y categorizar progresivamente desde Django Admin.

DECISIÓN DE DISEÑO 1.2: ERRADICACIÓN DE TIPOS FLOTANTES — PRECISIÓN FIJA NUMERIC

Bajo ninguna circunstancia se utilizan tipos IEEE 754 (`FLOAT`, `REAL` o `DOUBLE PRECISION`). En el comercio ferretero nicaragüense, los márgenes de ganancia y el IVA (15%) exigen precisión contable absoluta. Se establecieron dos escalas fijas:

- `NUMERIC(14,4)`: Para costos de compra, precios base y cálculos de IVA. Cuatro decimales protegen contra errores de redondeo acumulativo al prorratear fletes o descuentos por volumen.
- `NUMERIC(14,2)` y `NUMERIC(12,2)`: Para totales finales facturados, inventario físico y cantidades comerciales.

La versión 2.0 mantiene esta regla: `compra_items.costo_unitario`, `factura_items.precio_unitario_snapshot` y `costo_unitario_snapshot` usan `NUMERIC(14,4)`; los totales, descuentos, impuestos y montos de caja usan `NUMERIC(14,2)`.

DECISIÓN DE DISEÑO 1.3: EXTENSIONES POSTGRESQL (PG_TRGM Y CITEXT)

El DDL arranca ejecutando `CREATE EXTENSION IF NOT EXISTS pg_trgm`; y `CREATE EXTENSION IF NOT EXISTS citext`;

- `pg_trgm` permite búsquedas difusas mediante trigramas (índices GIN). En ferretería, los clientes suelen buscar "esmeril" o "taladro dewalt" con errores tipográficos; el índice `idx_productos_nombre_trgm` evalúa la similitud en milisegundos. En v2.0 se reutiliza también para buscar clientes por nombre en la caja (`idx_clientes_nombre_trgm`).
- `citext` proporciona tipos de texto que comparan caracteres sin distinguir mayúsculas ni minúsculas. Se aplica en `clientes.email`, `proveedores.email` y, desde v2.0, `empleados.email`.

1.4. Una sola base de datos para dos sistemas NUEVO v2.0

El proyecto dejó de ser solo una tienda en línea: ahora es una plataforma integrada por dos sistemas que comparten el mismo inventario, los mismos precios y los mismos clientes. **Next.js** atiende el e-commerce y **Django** atiende el panel de inventario, el POS de mostrador y el control monetario de ambos canales.

DECISIÓN DE DISEÑO 1.4: MODELO ÚNICO DE VENTAS EN LUGAR DE DOS MODELOS PARALELOS

La alternativa evaluada fue mantener `pedidos` para la web y crear tablas separadas (`ventas_pos`, `ventas_pos_items`) para el mostrador. Se descartó por tres razones: (1) todo reporte de ventas, IVA o utilidad tendría que unir dos fuentes y mantener dos lógicas de cálculo; (2) el stock y el Kardex se afectan igual sin importar el canal, así que separar las ventas obligaría a duplicar reglas; (3) un cliente que compra en mostrador y luego en línea generaría dos historiales.

La solución adoptada conserva `pedidos` como el documento **comercial** del e-commerce (carrito confirmado, dirección de envío, estado logístico) y introduce `facturas` como el documento **fiscal y contable** común a ambos canales. Un pedido web genera una factura; una venta de mostrador genera una factura directamente. Todo el análisis financiero sale de `facturas`.

2. CATÁLOGO Y ENTIDADES MAESTRAS (DDL DETALLADO)

2.1. Tabla categorías

Estructura jerárquica para la clasificación de productos.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador único numérico de 64 bits según estándar SQL:2003.
nombre	VARCHAR(150)	NOT NULL	Nombre de la categoría (ej. "Tornillería y Fijación").
slug	VARCHAR(160)	NOT NULL UNIQUE	Identificador URL-friendly indexado para rutas en Next.js (ej. /categoria/tornilleria).
categoria_padre_id	BIGINT	FK INDEX ON DELETE RESTRICT	Auto-referencia a categorias(id). Admite NULL si es categoría raíz (nivel 0).
nivel	SMALLINT	NOT NULL DEFAULT 0	Profundidad del nodo (0=Raíz, 1=Subcategoría, 2=Clase). Optimiza renderizado en frontend.
activo	BOOLEAN	NOT NULL DEFAULT TRUE	Bandera de baja lógica para ocultar ramas completas del árbol sin destruir datos.
Constraint	UNIQUE	(nombre, categoria_padre_id)	Clave compuesta única: Prohíbe dos categorías con el mismo nombre bajo el mismo padre, pero permite repetir nombres bajo padres distintos.

DECISIÓN DE DISEÑO 2.1: ÁRBOL AUTO-REFERENCIADO VS MODELO PLANO

En lugar de crear 3 tablas separadas (Categorías, Subcategorías, Clases) o 3 columnas fijas en producto, un árbol auto-referenciado con categoria_padre_id soporta N niveles de profundidad sin alterar la estructura física de la base de datos. El índice idx_categorias_padre garantiza que las consultas recursivas (Common Table Expressions - CTE) se ejecuten en tiempo logarítmico.

2.2. Tabla marcas

Registro de fabricantes y marcas comerciales distribuidas en la ferretería.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador primario de la marca.
nombre	VARCHAR(120)	NOT NULL UNIQUE	Nombre de la marca comercial (Stanley, DeWalt, Truper, Bosch, Holcim).
slug	VARCHAR(130)	NOT NULL UNIQUE	Identificador textual para filtrado directo en la tienda web (ej. /?marca=dewalt).
logo_url	TEXT	NULLABLE	Ruta hacia el activo gráfico del fabricante para carruseles de marca en portada.

2.3. Tabla unidades_medida

Catálogo de unidades de venta y control físico de existencias.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la unidad.
nombre	VARCHAR(60)	NOT NULL UNIQUE	Nombre completo (UNIDAD, GALON, PAR, METRO, KILOGRAMO, BOLSA).
abreviatura	VARCHAR(15)	NOT NULL	Símbolo estándar para recibos y fichas técnicas (und, gal, m, kg).

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
es_fraccionable	BOOLEAN	NOT NULL DEFAULT FALSE	Regla de negocio crítica: Autoriza o prohíbe la venta en números decimales.

DECISIÓN DE DISEÑO 2.3: LA LÓGICA DE FRACCIONAMIENTO FERRETERO

En retail general, casi todo se vende en enteros. En una ferretería coexisten artículos discretos (no se puede vender 0.5 martillos ni 1.3 serruchos) con artículos a granel (se despachan 2.5 kg de clavos de 2", 15.5 metros de cable THHN calibre 12, o 0.5 galones de thinner). El atributo es_fraccionable es consultado por Next.js para renderizar inputs numéricos de tipo step="0.01" vs step="1", y por Django para validar que nadie agregue fracciones de artículos indivisibles. En v2.0 la misma validación debe aplicarse en el POS al construir factura_items y en la recepción de compra_items.

2.4. Tabla proveedores

Entidades comerciales de aprovisionamiento y distribución mayorista. En v2.0 es referenciada por compras.proveedor_id.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de proveedor.
nombre	VARCHAR(200)	NOT NULL	Razón social de la empresa proveedora.
codigo_fiscal	VARCHAR(30)	NULLABLE	RUC o identificador tributario para emisión de órdenes de compra y retenciones de ley.
telefono	VARCHAR(30)	NULLABLE	Teléfono de contacto comercial.
email	CITEXT	NULLABLE	Correo de pedidos insensible a mayúsculas/minúsculas.
activo	BOOLEAN	NOT NULL DEFAULT TRUE	Permite inhabilitar compras a proveedores dados de baja.
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()	Fecha de registro en el sistema con huso horario explícito.

2.5. Tabla sucursales

Modelado de puntos de venta físicos y bodegas de despacho. En v2.0 es la referencia territorial de empleados, cajas, compras y facturas.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la sucursal.
nombre	VARCHAR(120)	NOT NULL	Nombre comercial de la tienda (reemplaza la columna plana "PDV" del Excel original).
direccion	TEXT	NULLABLE	Dirección física mostrada al cliente cuando elige la modalidad "Retiro en Tienda".
telefono	VARCHAR(30)	NULLABLE	Línea fija o móvil de atención al cliente de la tienda física.
es_bodega_ecommerce	BOOLEAN	NOT NULL DEFAULT FALSE	Bandera de orquestación: Especifica cuál sucursal centraliza el despacho a domicilio del e-commerce.
activo	BOOLEAN	NOT NULL DEFAULT TRUE	Permite cerrar temporalmente sucursales por inventario o reformas.

DECISIÓN DE DISEÑO 2.5: AISLAMIENTO DEL DESPACHO WEB CON ES_BODEGA_ECOMMERCE

Cuando un usuario realiza un pedido con envío a domicilio en Next.js, el backend necesita saber contra qué inventario físico descontar y desde qué punto sale el camión ferretero. En lugar de inventar una bodega virtual que distorsione la contabilidad, se marca una sucursal real con es_bodega_ecommerce = true, permitiendo que la tienda online opere con stock garantizado sin desabastecer cajas físicas imprevisiblemente.

3. ARTÍCULOS, VARIANTES Y MULTIMEDIA (DDL DETALLADO)

3.1. Tipo ENUM: estado_producto

```
CREATE TYPE estado_producto AS ENUM ('activo', 'inactivo', 'descontinuado');
```

Controla el ciclo de vida del ítem. `inactivo` lo oculta temporalmente; `descontinuado` bloquea nuevas órdenes de compra a proveedores mientras permite liquidar el stock remanente en piso de venta. Desde v2.0 esa regla se aplica al crear `compra_items` (validación en Django).

3.2. Tabla productos

Entidad central del modelo de negocio ferretero.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Clave subrogada artificial de alto rendimiento para joins internos.
codigo_producto	VARCHAR(30)	NOT NULL UNIQUE	Clave natural del negocio: El "Código Producto" original del Excel (SKU interno).
codigo_fabricante	VARCHAR(60)	NULLABLE	Part number del fabricante (ej. "DW-2401"). Utilizado en garantías oficiales.
codigo_fiscal	VARCHAR(30)	NULLABLE	Clasificación arancelaria centroamericana (SAC) para auditoría fiscal aduanera.
nombre	VARCHAR(250)	NOT NULL	Denominación técnica completa (ej. "Martillo Stanley 16oz Mango Fibra Vidrio").
slug	VARCHAR(260)	NOT NULL UNIQUE	URL canónica en Next.js (ej. /productos/martillo-stanley-16oz).
descripcion	TEXT	NULLABLE	Especificaciones técnicas, voltajes, dimensiones y compatibilidades de obra.
marca_id	BIGINT	FK INDEX NOT NULL	Referencia obligatoria a marcas(id). Todo producto pertenece a una marca.
categoria_id	BIGINT	FK INDEX NULLABLE	Referencia a categorias(id). Nullable para absorción del Excel sin errores.
unidad_medida_id	BIGINT	FK NOT NULL	Referencia a unidades_medida(id). Gobierna la regla de fraccionamiento.
proveedor_id	BIGINT	FK NULLABLE	Distribuidor habitual para generación de órdenes de compra (sugerido por defecto al crear una <code>compra</code>).
estado	estado_producto	INDEX DEFAULT 'activo'	Estado de vigencia del artículo.
stock_min MOD.	NUMERIC(12,2)	DEFAULT 0	Punto de reorden para disparar compras automáticas en Django. v2.0: también es el umbral que evalúa la vista <code>v_alerta_stock_bajo</code> .
stock_max	NUMERIC(12,2)	NULLABLE	Capacidad máxima de almacenaje físico para evitar sobrecostos de bodega.
publicado_ecommerce	BOOLEAN	NOT NULL DEFAULT FALSE	Filtro de seguridad: El admin decide qué artículos mostrar en la tienda web pública.
created_at / updated_at	TIMESTAMPZ	NOT NULL DEFAULT now()	Marcas de tiempo automáticas actualizadas mediante trigger PL/pgSQL.

3.3. Índices Estratégicos sobre productos

```
CREATE INDEX idx_productos_marca ON productos(marca_id);
CREATE INDEX idx_productos_categoria ON productos(categoria_id);
CREATE INDEX idx_productos_estado ON productos(estado);
CREATE INDEX idx_productos_nombre_trgm ON productos USING GIN (nombre gin_trgm_ops);
```

DECISIÓN DE DISEÑO 3.3: ÍNDICE GIN TRIGRAM PARA EL BUSCADOR WEB

En un catálogo de 7,000 productos con nombres largos y combinaciones de números y medidas ("Tornillo Hex 1/4 x 2"), una búsqueda estándar con LIKE '%termino%' ejecuta un escaneo secuencial (Seq Scan) que castiga al servidor de base de datos. El índice idx_productos_nombre_trgm utiliza el operador de trigramas de PostgreSQL para resolver búsquedas difusas instantáneamente por índice GIN invertido. El buscador del POS reutiliza el mismo índice.

3.4. Tabla producto_codigos_barra

Múltiples códigos de barras asociados a diferentes presentaciones de empaque.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador único del registro de código.
producto_id	BIGINT	FK ON DELETE CASCADE	Producto al que corresponde el código. Si se borra el producto, se eliminan sus códigos.
codigo_barras	VARCHAR(60)	NOT NULL UNIQUE	Cadena numérica o alfanumérica escaneable por lectores ópticos (EAN-13, UPC-A, Code 128).
presentacion	VARCHAR(40)	DEFAULT 'unidad'	Tipo de empaque asociado al código (unidad, caja_12, caja_100, bulto, pallet).

DECISIÓN DE DISEÑO 3.4: CARDINALIDAD 1 A N EN CÓDIGOS DE BARRA

El Excel original existía una sola columna de código de barras. Esto rompía la operación en caja física: cuando un cliente compra una caja entera de bombillos LED o un paquete de 100 tornillos, el cajero escanea el código exterior de la caja, no abre la caja para escanear una pieza individual. Al modelar una tabla 1:N con campo presentacion, el sistema POS de Elizabeth puede reconocer cualquier empaque y realizar la equivalencia correspondiente de forma inmediata. Nota v2.0: hoy **presentacion** es descriptivo; la equivalencia numérica (ej. caja = 12 unidades) todavía debe resolverla el backend (ver sección 16).

3.5. Tabla producto_imagenes

Galería multimedia para la presentación de productos en el catálogo de Next.js.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la imagen.
producto_id	BIGINT	FK ON DELETE CASCADE	Producto al que pertenece la imagen fotográfica.
url	TEXT	NOT NULL	URL del archivo multimedia almacenado en CDN, bucket S3 o servidor local.
orden	SMALLINT	NOT NULL DEFAULT 0	Prioridad de visualización: orden = 0 define la imagen principal de la tarjeta.
alt_text	VARCHAR(200)	NULLABLE	Texto alternativo para lectores de pantalla (accesibilidad web) y posicionamiento SEO.

4. PRECIOS, FISCALIDAD Y VISTA DE TARIFAS VIGENTES

4.1. Tabla producto_precios

Historial inmutable de precios y costos según el patrón Slowly Changing Dimensions (SCD Tipo 2).

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Clave primaria del registro histórico de precio.
producto_id	BIGINT	FK ON DELETE CASCADE	Producto sujeto a la tarifa.
costo_promedio MOD.	NUMERIC(14,4)	CHECK (costo_promedio >= 0)	Costo Promedio Ponderado (CPP) en libros contables tras compras de reposición. v2.0: se recalcula al recibir una compra (ver 4.4).
precio_sin_iva	NUMERIC(14,4)	CHECK (precio_sin_iva >= 0)	Precio base neto al público antes de impuestos.
porcentaje_iva	NUMERIC(5,2)	NOT NULL DEFAULT 0	Tasa impositiva (ej. 15.00% en Nicaragua, o 0% si el rubro goza de exención fiscal). Es la fuente que rige el IVA de cada venta.
precio_con_iva	NUMERIC(14,4)	STORED	Columna generada físicamente: (precio_sin_iva * (1 + porcentaje_iva / 100.0)).
vigente_desde	TIMESTAMP TZ	NOT NULL DEFAULT now()	Marca temporal de inicio de validez de este precio.
vigente_hasta	TIMESTAMP TZ	NULLABLE	Fin de validez. NULL representa el precio actualmente activo.
creado_por	VARCHAR(120)	NULLABLE	Auditoría: Identidad del usuario de Django Admin que autorizó la modificación tarifaria.

4.2. Índice Único Parcial: idxPrecioVigenteUnico

```
CREATE UNIQUE INDEX idxPrecioVigenteUnico ON producto_precios(producto_id) WHERE vigente_hasta IS NULL;
```

GARANTÍA MATEMÁTICA DE UNICIDAD TEMPORAL

¿Cómo garantiza la base de datos que jamás existan dos precios activos simultáneamente para el mismo producto? Un índice único estándar sobre producto_id impediría guardar el historial. La solución elegante es un índice parcial con la cláusula WHERE vigente_hasta IS NULL. Esto permite infinitas filas pasadas con fecha de cierre, pero prohíbe terminantemente insertar una segunda fila activa mientras la anterior no haya sido cerrada formalmente.

4.3. Vista de Acceso Rápido: vPrecioActual

```
CREATE VIEW vPrecioActual AS
SELECT producto_id, costo_promedio, precio_sin_iva, porcentaje_iva, precio_con_iva
FROM producto_precios WHERE vigente_hasta IS NULL;
```

Esta vista desacopla la complejidad del historial temporal respecto a las consultas del frontend. Cuando Next.js o Django REST Framework necesitan los precios vigentes para el catálogo, no ejecutan filtros manuales de fechas: consultan directamente la vista, garantizando velocidad y exactitud. **En v2.0 el POS también la consulta** para obtener precio y costo al armar una factura.

4.4. Relación con compras y facturación NUEVO v2.0

La versión 2.0 conecta el historial de precios con dos documentos nuevos:

- **Al recibir una compra**, el costo promedio ponderado se recalcula: $CPP_{nuevo} = (stock_{total} \times CPP_{actual} + cantidad \times costo_{compra}) / (stock_{total} + cantidad)$, usando el stock sumado de todas las sucursales. El backend cierra la fila vigente (`vigente_hasta = now()`) e inserta una nueva con el CPP actualizado y el mismo precio de venta, en la misma transacción. Hacerlo en una sola transacción es obligatorio: el índice parcial impediría insertar la nueva fila mientras la anterior siga abierta.
- **Al facturar**, `factura_items` copia el precio y el costo vigentes (`precio_unitario_snapshot`, `costo_unitario_snapshot`). Así la utilidad de una venta pasada queda fija aunque el CPP cambie después.

5. INVENTARIO MULTISUCURSAL Y KARDEX INMUTABLE

5.1. Tabla inventario

Saldo en caché de existencias por sucursal física para lecturas de alto tráfico.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador del registro de saldo.
producto_id	BIGINT	FK INDEX ON DELETE CASCADE	Producto inventariado.
sucursal_id	BIGINT	FK ON DELETE RESTRICT	Sucursal física que custodia físicamente las unidades.
stock	NUMERIC(14,2)	CHECK (stock >= 0)	Existencia física real contada en los anaqueles o bodegas.
comprometido	NUMERIC(14,2)	CHECK (comprometido >= 0)	Unidades apartadas por carritos web en proceso de pago o pedidos pendientes de retiro.
disponible	NUMERIC(14,2)	STORED	Columna generada físicamente: (stock - comprometido). Saldo apto para venta.
updated_at	TIMESTAMP TZ	NOT NULL DEFAULT now()	Timestamp de la última alteración de saldo.
Constraint	UNIQUE	(producto_id, sucursal_id)	Clave compuesta única: Un producto solo tiene una fila de saldo por tienda.
Constraint	CHECK	(comprometido <= stock)	Blindaje matemático: Impide que las reservas excedan la existencia física real.

Como el POS y el e-commerce descuentan de la misma tabla, el backend debe leer y modificar cada fila de `inventario` con bloqueo (`SELECT ... FOR UPDATE`) dentro de la transacción de venta. Los CHECK anteriores actúan como red de seguridad final: si dos ventas simultáneas intentan vender la última unidad, la segunda falla en lugar de dejar stock negativo.

5.2. Tipo ENUM: tipo_movimiento

```
CREATE TYPE tipo_movimiento AS ENUM ('entrada', 'venta', 'ajuste', 'reserva', 'liberacion_reserva', 'devolucion');
```

Sin cambios en v2.0: `entrada` cubre la recepción de compras y `venta` cubre tanto el POS como el e-commerce. Los traslados entre sucursales requerirían valores nuevos (ver sección 16).

5.3. Tabla movimientos_inventario (El Kardex Contable) MODIFICADO v2.0

Bitácora transaccional e inmutable de toda alteración de stock en la empresa.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Número de asiento contable correlativo e inmutable.
producto_id	BIGINT	FK INDEX NOT NULL	Artículo impactado. Sin CASCADE: Jamás se borra un movimiento histórico de inventario.
sucursal_id	BIGINT	FK NOT NULL	Sucursal donde ocurrió físicamente el movimiento.
tipo	tipo_movimiento	NOT NULL	Naturaleza de la operación (entrada por compra, venta POS, reserva web, etc.).
cantidad	NUMERIC(14,2)	NOT NULL	Monto algebraico de unidades (positivo para aumentos, negativo para salidas).
referencia	VARCHAR(80)	NULLABLE	Documento vinculante en texto legible (ej. Factura proveedor F-8491, Pedido CW-00123, Ticket POS-12). Se conserva como etiqueta humana.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
usuario MOD.	VARCHAR(120)	NULLABLE	Auditoría textual del operador o servicio (ej. "webhook-stripe"). v2.0: para operadores humanos se prefiere <code>empleado_id</code> ; el campo se conserva para servicios automáticos y registros previos.
created_at	TIMESTAMPZ	INDEX NOT NULL DEFAULT now()	Marca de tiempo inmutable para auditorías fiscales.
compra_id NUEVO	BIGINT	FK INDEX NULLABLE	Compra que originó la entrada de mercadería.
factura_id NUEVO	BIGINT	FK INDEX NULLABLE	Factura (POS o web) que originó la salida por venta o la devolución.
pedido_id NUEVO	BIGINT	FK NULLABLE	Pedido web que originó una reserva o su liberación.
empleado_id NUEVO	BIGINT	FK NULLABLE	Empleado que ejecutó el movimiento. NULL cuando lo genera un servicio automático.

DECISIÓN DE DISEÑO 5.3: LLAVES FORÁNEAS EN LUGAR DE TEXTO LIBRE COMO REFERENCIA

En v1.0 el vínculo entre un movimiento y su documento de origen era el campo `referencia`, un texto libre. Eso alcanzaba mientras solo existían pedidos web, pero con compras, facturas de mostrador y de e-commerce ya no es suficiente: un texto no permite hacer joins, no garantiza que el documento exista y no impide errores de tipeo. Con `compra_id`, `factura_id` y `pedido_id` la base de datos responde con integridad referencial a la pregunta "¿por qué cambió el stock de este producto?" y permite reconstruir el Kardex de una factura o de una compra con un simple JOIN. Se mantuvieron los tres campos como columnas independientes y nulables (en vez de una relación polimórfica genérica) porque PostgreSQL puede verificarlos con FK reales. No se impone que solo uno esté lleno: un pedido web genera una reserva con `pedido_id` y luego una venta con `factura_id` (y opcionalmente también `pedido_id`).

LA REGLA DE ORO DEL KARDEX: INVENTARIO ES LA CACHÉ, MOVIMIENTOS ES LA VERDAD

En sistemas comerciales robustos, la tabla inventario nunca es la autoridad final: es únicamente una tabla de saldos agregados para que Next.js consulte el stock disponible en 1 milisegundo sin tener que sumar millones de filas. La verdadera autoridad es `movimientos_inventario`. En caso de discrepancia física o auditoría tributaria, el saldo real de un producto se reconstruye sumando algebraicamente su historial de movimientos.

6. CLIENTES, DIRECCIONES Y CARRITO PERSISTENTE

6.1. Tabla clientes MODIFICADO v2.0

Cuentas de compradores del portal e-commerce y clientes registrados en mostrador (separadas del personal administrativo).

```
CREATE TYPE tipo_cliente AS ENUM ('persona', 'empresa');
```

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador único de cliente.
nombre	VARCHAR(150)	NOT NULL	Nombre y apellidos o razón social para facturación.
email MOD.	CITEXT	NULLABLE UNIQUE	Correo electrónico validado, insensible a mayúsculas/minúsculas. v2.0: antes NOT NULL; ahora opcional para clientes de mostrador. PostgreSQL permite varios NULL bajo un UNIQUE, por lo que no hay colisiones entre clientes sin correo.
telefono	VARCHAR(30)	NULLABLE	Teléfono para coordinación de entrega con motorizado o camión.
codigo_fiscal MOD.	VARCHAR(30)	NULLABLE INDEX	RUC de empresa o Cédula ciudadana para emisión de facturas formales con retención tributaria. v2.0: indexado para búsqueda rápida en caja.
password_hash MOD.	TEXT	NULLABLE	Hash criptográfico de contraseña (Argon2 / PBKDF2) para autenticación JWT en Next.js. v2.0: antes NOT NULL; NULL significa "cliente sin cuenta web".
activo	BOOLEAN	NOT NULL DEFAULT TRUE	Permite suspender temporalmente el acceso a la cuenta sin borrar el historial de pedidos.
created_at	TIMESTAMP TZ	NOT NULL DEFAULT now()	Fecha de registro en la tienda.
updated_at	TIMESTAMP TZ	NOT NULL DEFAULT now()	Marca de última modificación del perfil (mantenida por trigger PL/pgSQL).
tipo_cliente NUEVO	tipo_cliente	NOT NULL DEFAULT 'persona'	Distingue persona natural de empresa (constructoras, contratistas). Base para reportes y futuras condiciones comerciales.
origen NUEVO	VARCHAR(20)	NOT NULL DEFAULT 'ecommerce' CHECK IN ('ecommerce','pos')	Canal por el que se registró el cliente. Permite medir cuántos clientes nacen en mostrador vs. web.
Constraint NUEVO	CHECK	chk_cliente_login_requiere_email	Coherencia de acceso: si existe password_hash (cuenta web), el email es obligatorio, porque es el identificador de login.

Índices añadidos: idx_clientes_codigo_fiscal (búsqueda por RUC o cédula al facturar) e idx_clientes_nombre_trgm (búsqueda difusa por nombre en la caja).

DECISIÓN DE DISEÑO 6.1: UNA SOLA TABLA DE CLIENTES PARA MOSTRADOR Y WEB

Se evaluó crear una tabla clientes_pos separada. Se descartó porque un mismo comprador (por ejemplo, un maestro de obras) compra en mostrador y luego se registra en la tienda en línea: con dos tablas tendría dos historiales y sería imposible calcular su volumen real de compra. Con una sola tabla, cuando ese cliente crea su cuenta web basta con asignarle correo y contraseña a su registro existente.

Para ello fue necesario relajar email y password_hash a NULLABLE, y proteger el caso crítico con chk_cliente_login_requiere_email (no puede existir una cuenta con contraseña y sin correo). El comprador anónimo del mostrador ("consumidor final") no se registra como cliente: facturas.cliente_id queda en NULL, lo que evita crear un cliente comodín que contamine reportes.

codigo_fiscal se dejó deliberadamente sin restricción UNIQUE: una empresa puede tener varios contactos con el mismo RUC y un error de digitación no debe bloquear una venta en caja. La deduplicación se resuelve por proceso administrativo, no por bloqueo de base de datos.

6.2. Tabla direcciones_cliente

Libreta de destinos de envío para clientes registrados.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador del domicilio.
cliente_id	BIGINT	FK ON DELETE CASCADE	Cliente titular de la dirección.
alias	VARCHAR(60)	NULLABLE	Etiqueta descriptiva (ej. "Casa", "Taller", "Obra Carretera Sur").
direccion	TEXT	NOT NULL	Texto detallado de la dirección geográfica.
ciudad	VARCHAR(80)	NULLABLE	Municipio o departamento (Managua, Masaya, León, Granada).
referencia	TEXT	NULLABLE	Contexto geográfico nicaragüense: Puntos de referencia tradicionales esenciales para mensajería (ej. "De la rotonda 2c abajo, portón negro").
predeterminada	BOOLEAN	NOT NULL DEFAULT FALSE	Marca la dirección cargada por defecto al entrar al checkout.

6.3. Tablas carritos y carrito_items

Persistencia de canastas de compra tanto para clientes autenticados como para invitados anónimos.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
carritos.cliente_id	BIGINT	FK NULLABLE	Vinculado a clientes(id) si el comprador ha iniciado sesión.
carritos.session_id	VARCHAR(100)	NULLABLE	Token transmitido vía cabecera HTTP X-Session-Id para invitados no logueados.
carrito_items.cantidad	NUMERIC(12,2)	CHECK (cantidad > 0)	Unidades seleccionadas. Admite decimales según es_fraccionable.
carrito_items.UNIQUE	Constraint	(carrito_id, producto_id)	Garantiza que un producto no se duplique en varias filas dentro de la misma canasta.

7. PEDIDOS, COBROS Y SNAPSHOT DE FACTURACIÓN

7.1. Tipo ENUM: estado_pedido

```
CREATE TYPE estado_pedido AS ENUM ('pendiente', 'pagado', 'en_preparacion', 'enviado', 'entregado', 'cancelado');
```

7.2. Tabla pedidos

Documento formal de venta e-commerce que consolida la transacción. **v2.0:** al pasar a `pagado`, el pedido genera su factura (`facturas.pedido_id`, relación 1 a 1); ver sección 11.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador secuencial interno de clave primaria.
numero_pedido	VARCHAR(20)	NOT NULL UNIQUE	Código visible de cara al cliente: Folio alfanumérico formateado (ej. "CW-000123").
cliente_id	BIGINT	FK INDEX NOT NULL	Cliente titular de la compra.
sucursal_id	BIGINT	FK NOT NULL	Sucursal asignada para retiro en tienda o despacho vehicular.
direccion_envio_id	BIGINT	FK NULLABLE	Destino de flete. Permanece NULL si el cliente seleccionó retiro en mostrador.
estado	estado_pedido	INDEX DEFAULT 'pendiente'	Etapas de la orden en el ciclo operativo.
subtotal	NUMERIC(14,2)	NOT NULL	Monto total neto antes de impuestos en Córdoba.
iva_total	NUMERIC(14,2)	NOT NULL	Impuesto consolidado recaudado (15% ferretero).
total	NUMERIC(14,2)	NOT NULL	Importe bruto final debitado al comprador.
created_at / updated_at	TIMESTAMPZ	NOT NULL DEFAULT now()	Trazabilidad temporal con trigger automático.

7.3. Tabla pedido_items (Diseño Inmutable de Facturación)

Líneas de detalle con congelamiento de precios en el instante de la transacción.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la línea.
pedido_id	BIGINT	FK ON DELETE CASCADE	Pedido al que pertenece este artículo.
producto_id	BIGINT	FK ON DELETE RESTRICT	Artículo adquirido. RESTRICT estricto: Impide eliminar un producto si ya tiene facturación histórica.
cantidad	NUMERIC(12,2)	CHECK (cantidad > 0)	Volumen vendido.
precio_unitario_snapshot	NUMERIC(14,4)	NOT NULL	Precio congelado: Valor unitario pactado al segundo exacto de la compra.
iva_unitario_snapshot	NUMERIC(14,4)	NOT NULL	IVA unitario congelado.
subtotal	NUMERIC(14,2)	STORED	Columna calculada: (cantidad * precio_unitario_snapshot).

POR QUÉ UN JOIN DINÁMICO A PRODUCTO_PRECIOS DESTRUIRÍA LA CONTABILIDAD

Un error común de diseño novato es calcular el importe de un pedido pasado haciendo un JOIN a la tabla de productos o precios actuales. Si hoy un saco de cemento cuesta C\$ 360 y dentro de tres meses la inflación lo sube a C\$ 420, el JOIN recalcularía hacia arriba las facturas pasadas, alterando las declaraciones fiscales y los recibos de los clientes. El snapshot de precios en `pedido_items` garantiza inmutabilidad legal absoluta. El mismo principio rige en `factura_items` y `compra_items`.

7.4. Tabla pagos MODIFICADO v2.0

Registro de transacciones financieras y pasarelas de cobro electrónico. **v2.0:** ahora registra también los cobros de las facturas de mostrador.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador único del pago.
pedido_id MOD.	BIGINT	FK ON DELETE CASCADE NULLABLE	Pedido web que salda esta transacción. v2.0: antes NOT NULL; NULL en pagos de ventas de mostrador.
metodo_pago MOD.	VARCHAR(40)	NOT NULL CHECK IN (...)	Modalidad utilizada: efectivo , tarjeta (Stripe / POS), transferencia , efectivo_contraentrega . v2.0: restringido por CHECK a esos cuatro valores.
monto MOD.	NUMERIC(14,2)	NOT NULL CHECK (monto > 0)	Cantidad monetaria efectivamente aplicada a la deuda (no el billete entregado). v2.0: se añade CHECK de monto positivo.
estado	VARCHAR(30)	NOT NULL DEFAULT 'pendiente'	Transición de cobro: pendiente, aprobado, rechazado.
referencia_transaccion	VARCHAR(120)	NULLABLE	Identificador devuelto por la pasarela (ej. Stripe PaymentIntent ID: pi_3MtwL24... o voucher bancario).
created_at	TIMESTAMP TZ	NOT NULL DEFAULT now()	Marca de tiempo de procesamiento.
factura_id NUEVO	BIGINT	FK INDEX NULLABLE	Factura que salda este pago. Obligatorio en ventas de mostrador; en pedidos web se recomienda completarlo al emitir la factura.
sesion_caja_id NUEVO	BIGINT	FK NULLABLE	Turno de caja en el que se recibió el pago (solo POS). Permite cuadrar por turno cualquier método, no solo efectivo.
Constraint NUEVO	CHECK	chk_pago_destino: num_nonnulls(pedido_id, factura_id) >= 1	Blindaje: todo pago debe pertenecer a un pedido, a una factura o a ambos. Impide pagos huérfanos.

Índices añadidos: idx_pagos_pedido e idx_pagos_factura .

DECISIÓN DE DISEÑO 7.4: UNA SOLA TABLA DE PAGOS PARA LOS DOS CANALES

Por qué no una tabla pagos_pos : los cobros comparten estructura (método, monto, estado, referencia) y se analizan juntos en conciliación bancaria y cierres. Dos tablas obligarían a unir las en cada reporte financiero. Se relajó pedido_id a NULLABLE y se agregó factura_id , protegiendo la coherencia con chk_pago_destino .

Pagos mixtos: una venta de mostrador puede pagarse parte en efectivo y parte con tarjeta. Como pagos es 1:N respecto de la factura, se registra una fila por método; no hace falta un método "mixto".

Por qué monto es lo aplicado y no lo recibido: si el cliente entrega C\$ 100 por una factura de C\$ 65, el pago registra 65. El vuelto no se modela en la base de datos; de lo contrario el arqueo de caja mezclaría dinero del cliente con dinero del negocio.

Por qué CHECK y no ENUM para metodo_pago : la lista de métodos puede crecer (billetera móvil, cheque). Modificar un CHECK es un DROP/ADD CONSTRAINT , mientras que agregar valores a un ENUM tiene restricciones transaccionales en PostgreSQL. Como el campo ya existía como VARCHAR, evita además una conversión de tipo. Si pagos ya tiene filas con otros valores, deben normalizarse antes de aplicar el CHECK.

INTEGRACIÓN CON WEBHOOKS DE STRIPE Y TRANSICIONES DE ESTADO

El frontend en Next.js nunca marca un pedido como pagado de manera unilateral. Cuando el cliente paga con su tarjeta de prueba en Stripe, la pasarela emite un evento asíncrono seguro (webhook payment_intent.succeeded) hacia la API de Django. El backend valida la firma criptográfica del webhook, actualiza pagos.estado = 'aprobado', transiciona pedidos.estado = 'pagado' y autoriza el movimiento de inventario definitivo (de reserva a venta), liberando las piezas comprometidas. **En v2.0, en esa misma transacción se emite la factura del pedido** (canal 'ecommerce') copiando sus líneas a factura_items .

8. PARÁMETROS DEL NEGOCIO Y PERSONAL

8.1. Tabla parametros NUEVO v2.0

Configuración global del negocio. Es una tabla de **una sola fila**.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	SMALLINT	PK DEFAULT 1 CHECK (id = 1)	Clave fija: el CHECK garantiza que la tabla admita exactamente una fila.
nombre_negocio	VARCHAR(150)	NOT NULL DEFAULT 'Ferretería Construye'	Razón comercial impresa en facturas, recibos y correos.
codigo_fiscal	VARCHAR(30)	NULLABLE	RUC del negocio para el encabezado de las facturas.
direccion	TEXT	NULLABLE	Dirección fiscal impresa en documentos.
telefono	VARCHAR(30)	NULLABLE	Teléfono impreso en documentos.
moneda_base	CHAR(3)	NOT NULL DEFAULT 'NIO'	Código ISO 4217 de la moneda contable (Córdoba nicaragüense).
iva_porcentaje_default	NUMERIC(5,2)	NOT NULL DEFAULT 15.00	Tasa sugerida al crear un precio nuevo. No reemplaza a <code>producto_precios.porcentaje_iva</code> , que es lo que rige cada venta.
monto_max_caja	NUMERIC(14,2)	CHECK (>= 0) NULLABLE	Efectivo máximo permitido en una gaveta antes de exigir un retiro parcial. NULL = sin límite.
stock_bajo_alerta_default	NUMERIC(12,2)	NOT NULL DEFAULT 5	Umbral global usado por <code>v_alerta_stock_bajo</code> cuando un producto no define <code>stock_min</code> .
updated_at	TIMESTAMPZ	NOT NULL DEFAULT now()	Última modificación (mantenida por trigger).

DECISIÓN DE DISEÑO 8.1: TABLA DE FILA ÚNICA CON COLUMNAS TIPADAS

Las alternativas eran (a) un archivo `settings.py` y (b) una tabla clave-valor. La primera obliga a redespargar para cambiar, por ejemplo, el tope de caja; y no es visible desde SQL. La segunda pierde tipos y restricciones (todo sería texto). Una fila única con columnas tipadas permite editar desde Django Admin, validar con CHECK y consultar desde las vistas (`v_alerta_stock_bajo` lee `stock_bajo_alerta_default`).

El IVA por defecto (15%) es solo una comodidad de captura; el IVA aplicado a cada venta siempre sale del precio vigente del producto, de modo que los rubros exentos siguen funcionando.

8.2. Tabla empleados NUEVO v2.0

Perfil de recursos humanos del personal que opera el panel y el POS.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador del empleado.
auth_user_id	BIGINT	UNIQUE FK NULLABLE	Vínculo 1:1 con <code>auth_user</code> de Django (login y permisos). La FK se crea condicionalmente (ver Decisión 8.2).
nombres	VARCHAR(100)	NOT NULL	Nombres del empleado.
apellidos	VARCHAR(100)	NOT NULL	Apellidos del empleado.
cedula	VARCHAR(30)	NOT NULL UNIQUE	Cédula de identidad. Evita registrar dos veces a la misma persona.
telefono	VARCHAR(30)	NULLABLE	Teléfono de contacto.
email	CITEXT	NULLABLE	Correo del empleado, insensible a mayúsculas.
direccion	TEXT	NULLABLE	Domicilio del empleado.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
cargo	VARCHAR(80)	NULLABLE	Puesto informativo (Cajero, Bodeguero, Vendedor). No otorga permisos.
sucursal_id	BIGINT	FK INDEX NULLABLE	Sucursal a la que está asignado.
supervisor_id	BIGINT	FK ON DELETE SET NULL	Auto-referencia al jefe directo. Permite modelar el organigrama.
salario_base	NUMERIC(14,2)	NOT NULL DEFAULT 0 CHECK (>= 0)	Salario base mensual. Dato sensible.
bonos	NUMERIC(14,2)	NOT NULL DEFAULT 0 CHECK (>= 0)	Bonificaciones vigentes. Dato sensible.
fecha_ingreso	DATE	NOT NULL DEFAULT CURRENT_DATE	Fecha de contratación.
activo	BOOLEAN	NOT NULL DEFAULT TRUE	Baja lógica: un empleado que se va se desactiva, nunca se borra, porque sus facturas y movimientos lo referencian.
created_at / updated_at	TIMESTAMPZ	NOT NULL DEFAULT now()	Trazabilidad temporal (trigger updated_at).

DECISIÓN DE DISEÑO 8.2: PERFIL DE EMPLEADO SEPARADO DE LA AUTENTICACIÓN DE DJANGO

Se evitó guardar contraseña y rol en la tabla de personal, porque Django ya provee autenticación, hash de contraseñas y grupos de permisos (Administrador, Cajero, Bodeguero...). Duplicar contraseñas en otra tabla aumentaría la superficie de riesgo y obligaría a mantener dos sistemas de acceso. `empleados` guarda lo que Django no tiene (cédula, salario, ingreso, jefe) y se enlaza 1:1 por `auth_user_id`. Por eso `cargo` es solo informativo: **el acceso real lo define el grupo de Django**, no un texto libre.

FK condicional: `auth_user` lo crean las migraciones de Django. El script añade la FK solo si esa tabla ya existe. Si el script se ejecuta antes de las migraciones, la FK debe agregarse después con un `ALTER TABLE`. Si se usa un modelo de usuario personalizado, hay que ajustar el nombre de la tabla referenciada.

Confidencialidad: `salario_base` y `bonos` deben restringirse por permisos en Django Admin y **nunca** exponerse por la API que consume Next.js. El frontend público no debe tener ninguna ruta que lea `empleados`.

9. COMPRAS A PROVEEDORES

9.1. Tipo ENUM y tabla compras NUEVO v2.0

```
CREATE TYPE estado_compra AS ENUM ('pendiente', 'recibida', 'anulada');
```

Cabecera de cada orden de compra de mercadería.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la compra.
numero_compra	VARCHAR(20)	NOT NULL UNIQUE	Folio interno legible (ej. "OC-000045").
proveedor_id	BIGINT	FK INDEX NOT NULL	Proveedor al que se compra.
sucursal_id	BIGINT	FK NOT NULL	Sucursal cuyo inventario recibe la mercadería.
empleado_id	BIGINT	FK NOT NULL	Empleado que registró la compra.
numero_documento_proveedor	VARCHAR(60)	NULLABLE	Folio de la factura o remisión del proveedor, para conciliación.
fecha	DATE	INDEX NOT NULL DEFAULT CURRENT_DATE	Fecha del documento de compra.
estado	estado_compra	NOT NULL DEFAULT 'pendiente'	pendiente → recibida o anulada .
subtotal / iva_total / total	NUMERIC(14,2)	NOT NULL DEFAULT 0	Totales del documento, calculados por el backend al guardar.
observaciones	TEXT	NULLABLE	Notas libres (flete, acuerdos, incidencias).
created_at / updated_at	TIMESTAMP TZ	NOT NULL DEFAULT now()	Trazabilidad temporal (trigger updated_at).

9.2. Tabla compra_items NUEVO v2.0

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la línea.
compra_id	BIGINT	FK ON DELETE CASCADE	Compra a la que pertenece la línea.
producto_id	BIGINT	FK INDEX	Producto comprado.
cantidad	NUMERIC(12,2)	CHECK (cantidad > 0)	Unidades compradas. Debe respetar es_fraccionable .
costo_unitario	NUMERIC(14,4)	CHECK (costo_unitario >= 0)	Costo unitario sin IVA pactado con el proveedor. Cuatro decimales para prorrateos de flete y descuentos.
porcentaje_iva	NUMERIC(5,2)	NOT NULL DEFAULT 0	Tasa de IVA de la línea, congelada.
subtotal	NUMERIC(14,2)	STORED	Columna generada: (cantidad * costo_unitario).
Constraint	UNIQUE	(compra_id, producto_id)	Una sola línea por producto dentro de la misma compra.

DECISIÓN DE DISEÑO 9.1: RECEPCIÓN DE COMPRA COMO TRANSACCIÓN ÚNICA

Registrar una compra en estado `pendiente` no modifica inventario ni costos. Solo al marcarla `recibida` el backend ejecuta, en **una sola transacción**: (1) suma `inventario.stock` en la sucursal de la compra; (2) inserta en `movimientos_inventario` un movimiento `entrada` con `compra_id`; (3) recalcula el costo promedio ponderado y crea la nueva fila de `producto_precios` (sección 4.4). Si cualquier paso falla, no queda nada a medias.

Sin control de lote ni vencimiento: un tornillo o un martillo no caducan, y exigir esos campos obligaría al personal a llenar datos sin sentido. Si en el futuro se decide controlar productos con vida útil (pinturas, adhesivos, selladores), se puede agregar `lote` y `fecha_vencimiento` como columnas opcionales.

Los totales de la cabecera se guardan, no se recalculan, igual que en `pedidos`: un documento de compra ya conciliado con la factura del proveedor no debe cambiar si alguien edita una línea. La consistencia (cabecera = suma de líneas) la valida el backend; la base de datos no la impone. Esta versión no contempla recepción parcial ni compras a crédito (ver sección 16).

10. CAJAS Y CONTROL DE EFECTIVO

10.1. Tabla cajas NUEVO v2.0

Terminal física de cobro de una sucursal (por ejemplo "Caja 1" o "Caja Mostrador").

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la caja.
sucursal_id	BIGINT	FK NOT NULL	Sucursal donde está instalada.
nombre	VARCHAR(60)	NOT NULL	Nombre visible de la caja.
activa	BOOLEAN	NOT NULL DEFAULT TRUE	Permite retirar una caja sin borrar su historial de turnos.
Constraint	UNIQUE	(sucursal_id, nombre)	No puede haber dos cajas con el mismo nombre en una sucursal.

10.2. Tabla sesiones_caja (turnos) NUEVO v2.0

```
CREATE TYPE estado_sesion_caja AS ENUM ('abierta', 'cerrada');
```

Turno de trabajo de un cajero sobre una caja, desde la apertura hasta el arqueo de cierre.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador del turno.
caja_id	BIGINT	FK NOT NULL	Caja utilizada.
empleado_id	BIGINT	FK NOT NULL	Cajero responsable del turno.
monto_inicial	NUMERIC(14,2)	NOT NULL CHECK (>= 0)	Fondo de caja con el que se abre el turno.
fecha_apertura	TIMESTAMP TZ	NOT NULL DEFAULT now()	Momento de apertura.
fecha_cierre	TIMESTAMP TZ	NULLABLE	Momento de cierre. NULL mientras el turno esté abierto.
efectivo_esperado	NUMERIC(14,2)	NULLABLE	Efectivo que debería haber al cierre: <code>monto_inicial + SUM(movimientos_caja.monto)</code> . Se congela al cerrar.
monto_contado	NUMERIC(14,2)	NULLABLE	Efectivo que el cajero contó físicamente al cerrar.
diferencia	NUMERIC(14,2)	STORED	Columna generada: <code>(monto_contado - efectivo_esperado)</code> . Negativo = faltante; positivo = sobrante.
estado	estado_sesion_caja	NOT NULL DEFAULT 'abierta'	Estado del turno.
observaciones	TEXT	NULLABLE	Justificación del cajero ante una diferencia.
Constraint	CHECK	chk_sesion_cierre	Un turno <code>abierta</code> no puede tener <code>fecha_cierre</code> ; uno <code>cerrada</code> debe tenerla y debe traer <code>monto_contado</code> .
Índice único parcial	UNIQUE	idx_sesion_abierta_por_caja (caja_id) WHERE estado = 'abierta'	Una caja solo puede tener un turno abierto a la vez.
Índice único parcial	UNIQUE	idx_sesion_abierta_por_empleado (empleado_id) WHERE estado = 'abierta'	Un empleado solo puede tener un turno abierto a la vez.

10.3. Tabla movimientos_caja NUEVO v2.0

```
CREATE TYPE tipo_movimiento_caja AS ENUM
('venta_efectivo', 'devolucion_efectivo', 'ingreso_manual', 'egreso_manual', 'retiro_parcial');
```

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador del movimiento.
sesion_caja_id	BIGINT	FK INDEX NOT NULL	Turno al que pertenece el movimiento.
tipo	tipo_movimiento_caja	NOT NULL	Naturaleza del movimiento de efectivo.
monto	NUMERIC(14,2)	NOT NULL CHECK (monto <> 0)	Importe algebraico: positivo entra a la gaveta, negativo sale.
factura_id	BIGINT	FK NULLABLE	Factura que originó el movimiento (ventas y devoluciones).
pago_id	BIGINT	FK NULLABLE	Pago en efectivo que lo originó.
descripcion	VARCHAR(250)	NULLABLE	Motivo (obligatorio por proceso en ingresos y egresos manuales).
empleado_id	BIGINT	FK NULLABLE	Quién autorizó el movimiento.
created_at	TIMESTAMPZ	INDEX NOT NULL DEFAULT now()	Momento del movimiento.

DECISIÓN DE DISEÑO 10.1: TERMINAL Y TURNO SON COSAS DISTINTAS

Una sola tabla que represente a la vez la terminal y el turno no permite identificar la terminal. Con varias cajas por sucursal esa mezcla impide responder "¿cuánto vendió la Caja 2 esta semana?" o "¿qué cajero la tuvo el martes?". Separar **cajas** (la terminal, estable) de **sesiones_caja** (el turno, efímero) resuelve ambas preguntas y permite auditar por terminal y por persona.

DECISIÓN DE DISEÑO 10.2: EL KARDEX DEL EFECTIVO — SIN CONTADOR ACUMULADO

Una solución común es guardar un contador de ventas en efectivo que se incrementa con cada venta. Un contador cacheado así tiene dos problemas: se desincroniza si una venta se anula o se corrige, y sufre condiciones de carrera cuando dos operaciones lo actualizan a la vez. Es exactamente el problema que el Kardex resolvió para el inventario.

Por eso se aplica el mismo patrón: **movimientos_caja** es la **fuentes de verdad** (asientos algebraicos que nunca se editan) y el efectivo esperado se **calcula** como **monto_inicial + SUM(monto)**. Solo al cerrar se congela ese valor en **sesiones_caja.efectivo_esperado**, para que un arqueio cerrado no cambie si después se corrige un dato. **diferencia** es columna generada para que nadie pueda escribir un faltante "a mano".

DECISIÓN DE DISEÑO 10.3: QUÉ VA A CAJA Y CUÁNDO SE PIDE UN RETIRO

Solo el **efectivo** pasa por **movimientos_caja**. Los pagos con tarjeta o transferencia quedan en **pagos** (ligados al turno por **sesion_caja_id**) porque no incrementan la gaveta, pero sí deben cuadrar por turno en el reporte.

parametros.monto_max_caja funciona como política blanda: después de cada venta en efectivo, el backend compara el efectivo acumulado del turno contra ese tope y, si lo supera, exige un **retiro_parcial** (movimiento negativo). Esto reduce el riesgo de robo por exceso de efectivo en caja. La base de datos no bloquea la venta; solo alerta.

11. FACTURACIÓN UNIFICADA (POS Y E-COMMERCE)

11.1. Tablas facturas NUEVO v2.0

```
CREATE TYPE canal_venta AS ENUM ('pos', 'ecommerce');
CREATE TYPE estado_factura AS ENUM ('emitida', 'anulada');
```

Documento fiscal y contable común a las ventas de mostrador y a los pedidos web pagados.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador interno.
numero_factura	VARCHAR(30)	NOT NULL UNIQUE	Consecutivo de la factura. El formato y la serie legales deben confirmarse con contabilidad.
canal	canal_venta	NOT NULL	Origen de la venta: <code>pos</code> (mostrador) o <code>ecommerce</code> (pedido web).
fecha	TIMESTAMPZ	INDEX NOT NULL DEFAULT now()	Momento de emisión.
sucursal_id	BIGINT	FK NOT NULL	Sucursal que realiza la venta o despacha el pedido.
empleado_id	BIGINT	FK NULLABLE	Cajero que emitió la factura. Obligatorio en POS.
sesion_caja_id	BIGINT	FK INDEX NULLABLE	Turno de caja. Obligatorio en POS; prohibido en e-commerce.
pedido_id	BIGINT	UNIQUE FK NULLABLE	Pedido web de origen. UNIQUE : un pedido genera como máximo una factura.
cliente_id	BIGINT	FK INDEX NULLABLE	Cliente. NULL = consumidor final (solo permitido en POS).
cliente_nombre	VARCHAR(150)	NULLABLE	Nombre del cliente congelado al emitir.
cliente_codigo_fiscal	VARCHAR(30)	NULLABLE	RUC o cédula congelado al emitir.
subtotal	NUMERIC(14,2)	NOT NULL	Suma de líneas sin IVA, ya con descuentos aplicados.
descuento_total	NUMERIC(14,2)	NOT NULL DEFAULT 0	Suma de descuentos de línea (informativo).
iva_total	NUMERIC(14,2)	NOT NULL	Suma de <code>factura_items.impuesto</code> .
total	NUMERIC(14,2)	NOT NULL	<code>subtotal + iva_total</code> . Importe a cobrar.
estado	estado_factura	NOT NULL DEFAULT 'emitida'	<code>emitida</code> o <code>anulada</code> .
anulada_at / anulada_por / motivo_anulacion	TIMESTAMPZ / BIGINT / TEXT	NULLABLE	Quién anuló, cuándo y por qué. <code>anulada_por</code> referencia a <code>empleados</code> .
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()	Registro de creación.
Constraint	CHECK	chk_factura_canal	Coherencia por canal : POS ⇒ tiene sesión de caja y cajero, y no tiene pedido. E-commerce ⇒ tiene pedido y cliente, y no tiene sesión de caja.
Constraint	CHECK	chk_factura_anulacion	Una factura anulada debe registrar fecha y motivo de anulación.

Índices: `idx_facturas_fecha`, `idx_facturas_cliente`, `idx_facturas_sesion` e `idx_facturas_sucursal_canal` (`sucursal_id`, `canal`).

11.2. Tabla factura_items NUEVO v2.0

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
id	BIGINT	PK ALWAYS AS IDENTITY	Identificador de la línea.

Columna	Tipo de Dato	Restricciones	Descripción y Justificación Técnica
factura_id	BIGINT	FK INDEX ON DELETE CASCADE	Factura a la que pertenece.
producto_id	BIGINT	FK INDEX	Producto vendido. Sin CASCADE: no se puede borrar un producto con ventas.
cantidad	NUMERIC(12,2)	CHECK (cantidad > 0)	Volumen vendido.
precio_unitario_snapshot	NUMERIC(14,4)	NOT NULL CHECK (>= 0)	Precio unitario sin IVA congelado.
costo_unitario_snapshot	NUMERIC(14,4)	NOT NULL DEFAULT 0	Costo promedio vigente al vender. Permite calcular la utilidad real de cada línea.
descuento	NUMERIC(14,2)	NOT NULL DEFAULT 0 CHECK (>= 0)	Descuento en monto aplicado a toda la línea.
porcentaje_iva_snapshot	NUMERIC(5,2)	NOT NULL DEFAULT 0	Tasa de IVA congelada (0 para rubros exentos).
subtotal	NUMERIC(14,2)	STORED	Columna generada: (cantidad * precio_unitario_snapshot - descuento). Base imponible.
impuesto	NUMERIC(14,2)	STORED	Columna generada: ROUND((cantidad * precio - descuento) * porcentaje_iva / 100, 2).
Constraint	CHECK	chk_descuento_no_excede	El descuento no puede superar el valor bruto de la línea.

DECISIÓN DE DISEÑO 11.1: UNA FACTURA PARA AMBOS CANALES, PROTEGIDA POR UN CHECK

facturas unifica los dos canales, pero un modelo unificado corre el riesgo de aceptar filas incoherentes (una venta de mostrador sin caja, o una venta web sin pedido). **chk_factura_canal** lo impide directamente en la base de datos: cada canal exige unos campos y prohíbe otros. Así, cualquier bug en Django o Next.js que intente guardar una factura inválida es rechazado por el motor.

Relación con pedidos: el pedido sigue siendo el documento comercial y logístico (estados, dirección, entrega); la factura es el documento fiscal. **pedido_id UNIQUE** garantiza que un pedido no se facture dos veces.

DECISIÓN DE DISEÑO 11.2: DATOS FISCALES CONGELADOS Y ANULACIÓN SIN BORRADO

cliente_nombre y **cliente_codigo_fiscal** copian los datos del cliente al emitir. Es el mismo principio que el snapshot de precios: si el cliente cambia su nombre o su RUC en su perfil, la factura ya emitida no debe cambiar.

Una factura **nunca se borra**: se anula (**estado = 'anulada'**) dejando quién, cuándo y por qué (**chk_factura_anulacion**). La anulación debe generar en el mismo acto los movimientos inversos: un **movimientos_inventario** de tipo **devolucion** por cada línea y, si hubo efectivo, un **movimientos_caja** de tipo **devolucion_efectivo** . Sin esa reversa, el stock y la caja quedarían descuadrados.

DECISIÓN DE DISEÑO 11.3: FACTURA_ITEMS GUARDA PORCENTAJE DE IVA, NO IVA UNITARIO

Atención al copiar líneas de **pedido_items** a **factura_items**: los dos snapshots no tienen la misma forma. **pedido_items** guarda **iva_unitario_snapshot** (un monto por unidad), mientras que **factura_items** guarda **porcentaje_iva_snapshot** (un porcentaje). Al emitir la factura de un pedido web, el backend debe convertir: **porcentaje = iva_unitario / precio_unitario × 100** (con protección para precio 0).

Se eligió porcentaje en la factura porque la línea admite descuento: el IVA se calcula sobre la base **después** del descuento, y un IVA unitario fijo dejaría de ser exacto. Con el porcentaje congelado y el impuesto como columna generada, el cálculo es siempre reproducible.

Cuadre de redondeo: el impuesto se redondea a 2 decimales **por línea**. Por eso **facturas.iva_total** debe ser la **suma** de **factura_items.impuesto** , y no recalcarse sobre el subtotal, o aparecerían diferencias de centavos. Los pedidos web no manejan descuento, por lo que sus facturas lo llevan en 0.

DECISIÓN DE DISEÑO 11.4: COSTO CONGELADO PARA MEDIR UTILIDAD

costo_unitario_snapshot guarda el costo del producto al momento de vender. Con él, la utilidad de cualquier venta se calcula como **subtotal - cantidad × costo_unitario_snapshot** , sin depender de que el costo promedio siga siendo el mismo hoy. Sin ese snapshot, una compra posterior a un costo mayor alteraría retroactivamente el margen de ventas pasadas. Como el sistema ahora lleva el control monetario de ambos canales, esta métrica es imprescindible para el administrador.

12. VISTAS DE REPORTE

12.1. v_alerta_stock_bajo NUEVO v2.0

```
CREATE VIEW v_alerta_stock_bajo AS
SELECT p.id AS producto_id, p.codigo_producto, p.nombre,
       i.sucursal_id, i.stock, i.disponible,
       COALESCE(NULLIF(p.stock_min, 0),
                (SELECT stock_bajo_alerta_default FROM parametros WHERE id = 1)) AS umbral
FROM productos p
JOIN inventario i ON i.producto_id = p.id
WHERE p.estado = 'activo'
      AND i.stock <= COALESCE(NULLIF(p.stock_min, 0),
                              (SELECT stock_bajo_alerta_default FROM parametros WHERE id = 1));
```

Lista los productos que llegaron a su punto de reorden. Cada producto puede tener su propio punto de reorden (`stock_min`) y, si no lo define, se usa el umbral global de `parametros` . La evaluación es **por sucursal**, porque una tienda puede estar en cero mientras otra tiene de sobra.

LIMITACIÓN CONOCIDA DE LA VISTA

`stock_min` tiene DEFAULT 0, así que el valor 0 se interpreta como "no configurado" y cae al umbral global (`NULLIF(stock_min, 0)`). Un producto al que se quiera alertar realmente en 0 unidades no puede expresarse con este diseño. Se aceptó porque es más útil que cada producto nuevo, con `stock_min` en 0, no genere alertas nunca. Si el negocio necesita distinguir ambos casos, `stock_min` debería pasar a NULLABLE sin DEFAULT.

12.2. v_ventas_diarias NUEVO v2.0

```
CREATE VIEW v_ventas_diarias AS
SELECT (fecha AT TIME ZONE 'America/Managua')::date AS dia,
       sucursal_id, canal,
       COUNT(*) AS num_facturas,
       SUM(total) AS total_vendido
FROM facturas
WHERE estado = 'emitida'
GROUP BY 1, 2, 3;
```

Entrega, en una sola consulta, las ventas por día, sucursal y canal (mostrador vs. web), excluyendo facturas anuladas. `total_vendido` incluye IVA.

POR QUÉ SE CONVIERTE LA ZONA HORARIA ANTES DE TRUNCAR LA FECHA

Las fechas se guardan en TIMESTAMPTZ (internamente en UTC). Managua está en UTC-6 sin horario de verano, así que una venta hecha a las 8:00 p. m. hora local ocurre a las 2:00 a. m. UTC del día siguiente. Si la vista agrupara directamente por la fecha UTC, esa venta se contaría en el día equivocado y los cierres diarios no coincidirían con la realidad del negocio. `AT TIME ZONE 'America/Managua'` corrige eso. Es una vista normal (no materializada): calcula al consultar. Si el volumen de facturas crece mucho, conviene materializarla.

13. AUTOMATIZACIONES EN BASE DE DATOS: TRIGGERS PL/PGSQL

Para asegurar que los atributos de auditoría temporal (`updated_at`) no dependan de la memoria del programador ni puedan ser omitidos por consultas manuales desde herramientas como DBeaver o PgAdmin, la lógica de refresco se ejecuta a nivel de motor mediante un disparador nativo en lenguaje PL/pgSQL:

```
-- 1. Función disparadora que sobrescribe NEW.updated_at con la marca actual
CREATE OR REPLACE FUNCTION fn_set_updated_at() RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Triggers BEFORE UPDATE (esquema base)
CREATE TRIGGER trg_productos_updated_at BEFORE UPDATE ON productos
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();
CREATE TRIGGER trg_pedidos_updated_at BEFORE UPDATE ON pedidos
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();
CREATE TRIGGER trg_clientes_updated_at BEFORE UPDATE ON clientes
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();

-- 3. Triggers añadidos en v2.0 (reutilizan la misma función)
CREATE TRIGGER trg_empleados_updated_at BEFORE UPDATE ON empleados
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();
CREATE TRIGGER trg_compras_updated_at BEFORE UPDATE ON compras
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();
CREATE TRIGGER trg_parametros_updated_at BEFORE UPDATE ON parametros
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();
```

POR QUÉ UN TRIGGER DE BASE DE DATOS SUPERA AL `AUTO_NOW=TRUE` DEL ORM DE DJANGO

Django posee la opción `auto_now=True` en sus modelos. Sin embargo, este mecanismo de software tiene una debilidad conocida: solo se ejecuta cuando se llama al método `instance.save()` de Python. Si un desarrollador o administrador ejecuta un `Model.objects.filter(...).update(...)` masivo o corre un script SQL directo en el servidor de base de datos, Django no dispara el `auto_now`. El trigger nativo de PostgreSQL en `schema_construyeweb.sql` actúa a nivel de almacenamiento físico (Before Update), garantizando que jamás exista una fila modificada sin su timestamp real.

TABLAS SIN `UPDATED_AT` A PROPÓSITO

`facturas`, `factura_items`, `movimientos_inventario`, `movimientos_caja` y `compra_items` no llevan `updated_at` ni trigger: son registros contables que se insertan y no se editan. Lo único que cambia en una factura es su estado de anulación, que ya queda auditado con `anulada_at`, `anulada_por` y `motivo_anulacion`.

14. FLUJOS TRANSACCIONALES DE PUNTA A PUNTA

Las tablas por sí solas no garantizan que inventario, caja y facturas queden coherentes: eso depende de que el backend ejecute cada operación **dentro de una única transacción** (`transaction.atomic()` en Django). Si cualquier paso falla, se revierte todo. Estos son los cuatro flujos centrales.

14.1. Venta de mostrador (POS) NUEVO v2.0

1. Verificar que el cajero tenga una `sesiones_caja` en estado `abierta`.
2. Bloquear con `SELECT ... FOR UPDATE` las filas de `inventario` de los productos y la sucursal, y validar que `disponible` alcance.
3. Insertar la `facturas` (canal `pos`, con `sesion_caja_id` y `empleado_id`) y sus `factura_items`, copiando precio, costo e IVA desde `vprecio_actual`.
4. Insertar uno o más `pagos` (`factura_id`, `sesion_caja_id`); un pago mixto genera varias filas.
5. Por cada pago en efectivo, insertar un `movimientos_caja` de tipo `venta_efectivo` con monto positivo.
6. Por cada línea, descontar `inventario.stock` e insertar un `movimientos_inventario` de tipo `venta` (cantidad negativa) con `factura_id`.

14.2. Pedido del e-commerce NUEVO v2.0

1. Al confirmar el carrito se crea el `pedido` en estado `pendiente` y se reserva stock: `movimientos_inventario` tipo `reserva` (con `pedido_id`) e incremento de `inventario.comprometido`.
2. Stripe emite el webhook `payment_intent.succeeded`; Django valida la firma y marca `pagos.estado = 'aprobado'` y `pedidos.estado = 'pagado'`.
3. En la misma transacción se emite la `facturas` (canal `ecommerce`, con `pedido_id`), copiando `pedido_items` a `factura_items` con la conversión de IVA de la Decisión 11.3.
4. Cada línea pasa de reserva a venta: baja `stock` y `comprometido`, y se registra un movimiento `venta` con `factura_id`.
5. Si el pedido se cancela antes del pago, se registra `liberacion_reserva` y se devuelve el `comprometido`.

14.3. Compra a proveedor NUEVO v2.0

1. Crear la `compras` en estado `pendiente` con sus `compra_items`. No se afecta inventario.
2. Al recibir la mercadería, en una transacción: sumar `inventario.stock`, insertar movimientos `entrada` con `compra_id`, cerrar la fila vigente de `producto_precios` e insertar la nueva con el costo promedio recalculado, y marcar la compra como `recibida`.

14.4. Apertura y cierre de caja NUEVO v2.0

1. **Apertura:** se inserta la `sesiones_caja` con `monto_inicial`. Los índices únicos parciales impiden abrir un segundo turno en la misma caja o para el mismo cajero.
2. **Durante el turno:** cada venta en efectivo, ingreso, egreso o retiro parcial es un asiento en `movimientos_caja`.
3. **Cierre:** el sistema calcula `efectivo_esperado = monto_inicial + SUM(monto)`, el cajero registra `monto_contado`, y al pasar el turno a `cerrada` la columna `diferencia` queda calculada automáticamente.

REGLAS QUE HOY VIVEN EN EL BACKEND Y NO EN LA BASE DE DATOS

La base de datos impide los estados incoherentes más graves (dos turnos abiertos, facturas de canal inválido, pagos huérfanos, stock negativo), pero **no** impide, por ejemplo, insertar una factura POS en un turno ya cerrado, ni verifica que los totales de cabecera coincidan con la suma de sus líneas. Esas reglas deben validarse en Django. En la sección 16 se propone endurecerlas con triggers si el equipo lo considera conveniente.

15. MATRIZ CONSOLIDADA DE TABLAS Y RELACIONES

Tabla	Módulo Operativo	Claves Foráneas (FK)	Propósito Principal en ConstruyeWeb
categorias	Catálogo Maestro	categoria_padre_id → categorias	Árbol taxonómico de clasificación ferretera con profundidad dinámica.
marcas	Catálogo Maestro	—	Fabricantes de herramientas y materiales (Stanley, DeWalt, Truper, etc.).
unidades_medida	Catálogo Maestro	—	Control de venta discreta (enteros) vs materiales fraccionables (kilos, metros).
proveedores	Cadena de Suministro	—	Empresas mayoristas para reposición de inventario y órdenes de compra.
sucursales	Infraestructura Física	—	Locales de venta física, bodegas de stock y despacho e-commerce.
productos	Núcleo de Inventario	marca_id, categoria_id, unidad_medida_id, proveedor_id	Entidad central del catálogo maestro ferretero con SKU único.
producto_codigos_barra	Presentación y POS	producto_id → productos	Soporte multi-código para unidad suelta, caja y pallet en lectores ópticos.
producto_imagenes	Experiencia E-Commerce	producto_id → productos	Galería fotográfica con orden de visualización y metadatos SEO.
producto_precios	Finanzas e Impuestos	producto_id → productos	Historial inmutable de precios y CPP (SCD Tipo 2) con índice parcial.
inventario	Control Multicanal	producto_id → productos, sucursal_id → sucursales	Saldo rápido en caché de stock, reservas y unidades disponibles para venta (POS y web).
movimientos_inventario	Auditoría y Kardex	producto_id, sucursal_id, compra_id, factura_id, pedido_id, empleado_id	Bitácora inmutable de entradas, ventas, mermas y reservas contables, con trazabilidad al documento origen.
clientes	Compradores Web y Mostrador	—	Cuentas de retail electrónico y clientes de mostrador, desacopladas de administradores Django.
direcciones_cliente	Logística de Flete	cliente_id → clientes	Libreta de destinos de envío con puntos de referencia tradicionales.
carritos / items	Flujo de Compra	cliente_id → clientes, producto_id → productos	Canastas persistentes para clientes logueados o invitados anónimos.
pedidos	Ventas E-Commerce	cliente_id → clientes, sucursal_id → sucursales, direccion_envio_id → direcciones	Cabecera consolidada de la orden web con folio formal (CW-XXXXXX).
pedido_items	Ventas E-Commerce	pedido_id → pedidos, producto_id → productos	Líneas de pedido con congelamiento inmutable de precios pactados (Snapshot).
pagos	Pasarela y Cobros	pedido_id → pedidos, factura_id → facturas, sesion_caja_id → sesiones_caja	Liquidación financiera de pedidos web (Stripe Test Mode) y de ventas de mostrador, con pagos mixtos.
parametros	Configuración	—	Fila única con IVA por defecto, moneda, tope de caja y umbral global de stock bajo.
empleados	Personal y Seguridad	auth_user_id → auth_user, sucursal_id → sucursales, supervisor_id → empleados	Perfil de RR.HH. ligado al usuario de Django; jerarquía y datos salariales.
compras	Cadena de Suministro	proveedor_id → proveedores, sucursal_id → sucursales, empleado_id → empleados	Cabecera de órdenes de compra de mercadería a proveedores.
compra_items	Cadena de Suministro	compra_id → compras, producto_id → productos	Líneas de compra con costo unitario congelado.

Tabla	Módulo Operativo	Claves Foráneas (FK)	Propósito Principal en ConstruyeWeb
cajas	Control Monetario	sucursal_id → sucursales	Terminales físicas de cobro por sucursal.
sesiones_caja	Control Monetario	caja_id → cajas, empleado_id → empleados	Turnos de cajero con fondo inicial, arqueo y diferencia.
movimientos_caja	Control Monetario	sesion_caja_id → sesiones_caja, factura_id → facturas, pago_id → pagos, empleado_id → empleados	Kardex del efectivo: ventas, devoluciones, ingresos, egresos y retiros.
facturas	Ventas y Facturación	sucursal_id, empleado_id, sesion_caja_id, pedido_id, cliente_id	Documento fiscal unificado de ventas físicas y web.
factura_items	Ventas y Facturación	factura_id → facturas, producto_id → productos	Líneas facturadas con precio, costo, descuento e IVA congelados.

16. LIMITACIONES CONOCIDAS Y HOJA DE RUTA

La versión 2.0 cubre el flujo esencial de la ferretería, pero varios temas propios de este tipo de negocio quedaron **fuera de alcance a propósito** para no inflar el modelo antes de validar los procesos con el equipo. Se listan para que se decida cuáles priorizar.

Tema	Situación actual	Propuesta
Ventas a crédito (cuentas por cobrar)	<code>metodo_pago</code> no admite crédito; toda venta se cobra al momento.	Las ferreterías venden a crédito a constructores y contratistas. Agregar <code>cuentas_por_cobrar</code> y <code>abonos</code> , y el método <code>credito</code> .
Compras a crédito (cuentas por pagar)	<code>compras</code> no registra condición de pago ni vencimiento.	Agregar <code>condicion_pago</code> , <code>fecha_vencimiento_pago</code> y una tabla de pagos a proveedores.
Entrega a domicilio vs. retiro en tienda	El retiro se deduce de <code>direccion_envio_id = NULL</code> ; el costo de envío no está en los totales.	Agregar <code>tipo_entrega</code> y <code>costo_envio</code> a <code>pedidos</code> , y reflejarlo en la factura.
Traslados entre sucursales	<code>tipo_movimiento</code> no distingue traslados.	Agregar <code>traslado_salida</code> / <code>traslado_entrada</code> y una tabla <code>traslados</code> .
Devoluciones y notas de crédito	Solo se puede anular la factura completa.	Agregar <code>notas_credito</code> para devoluciones parciales.
Cotizaciones	No existen.	Muy usadas por constructores; agregar <code>cotizaciones</code> y <code>cotizacion_items</code> convertibles en factura.
Recepción parcial de compras	<code>compras</code> es todo o nada (<code>pendiente</code> → <code>recibida</code>).	Agregar estado <code>recibida_parcial</code> o una tabla de recepciones.
Equivalencias de presentación	<code>presentacion</code> es texto descriptivo.	Agregar <code>unidades_por_presentacion</code> (ej. caja = 12) en <code>producto_codigos_barra</code> para que el POS descuente el stock exacto.
Inmutabilidad real del Kardex y facturas	La "inmutabilidad" se respeta por convención; el motor no la impone.	Revocar <code>UPDATE</code> y <code>DELETE</code> sobre <code>movimientos_inventario</code> , <code>movimientos_caja</code> y <code>facturas</code> al rol de la aplicación.
Reglas hoy solo en el backend	Factura en turno cerrado; cabecera = suma de líneas; <code>pagos.estado</code> como texto libre.	Triggers de validación y un <code>CHECK</code> sobre <code>pagos.estado</code> si se quiere garantía en base de datos.
Historial salarial	<code>salario_base</code> y <code>bonos</code> se sobrescriben.	Si RR.HH. lo requiere, aplicar el mismo patrón SCD Tipo 2 de <code>producto_precios</code> .